

Deploy WordPress on an EC2 Instance

Overview

- This guide shows you how to set up WordPress on an Amazon EC2 instance.
- It uses the LAMP stack (Linux, Apache, MariaDB, PHP), which provides the web server, database, and scripting language that WordPress needs.
- The steps are written in order and include the exact commands you should run after connecting to your EC2 instance through SSH.

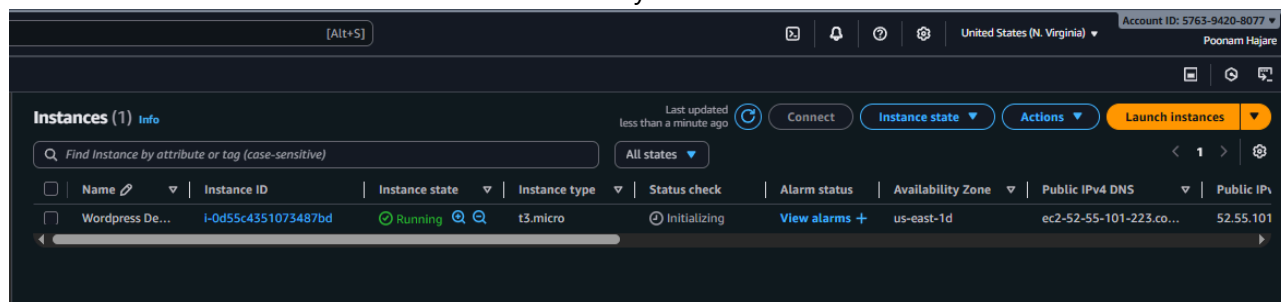
Prerequisites

1. An **AWS Account** and appropriate permission
2. A **key pair** (.pem file) for SSH access.
3. Security Group inbound rules:
 - SSH (TCP 22) — from your IP.
 - HTTP (TCP 80) — from 0.0.0.0/0.

Steps

1. Launch EC2 Instance

- Click **Instances** → then click **Launch Instances**.
- Name and tags → give name: **Wordpress Deployment**
- **Choose AMI** → select **Amazon Linux 2 AMI**.
- Instance type → **t2.micro or t3.micro**.
- **Key pair** → select or create one (.pem file will be downloaded).
- **Security group** → allow:
 - SSH (22) → from your IP.
 - HTTP (80) → from 0.0.0.0/0.
- Click **Launch Instance**.
- You can see now the instance is created successfully.



2. Connect to your EC2 instance

- Connect to your EC2 instance using SSH:

```
ssh -i "north-v-key2.pem" ec2-user@<PUBLIC_IP>
```

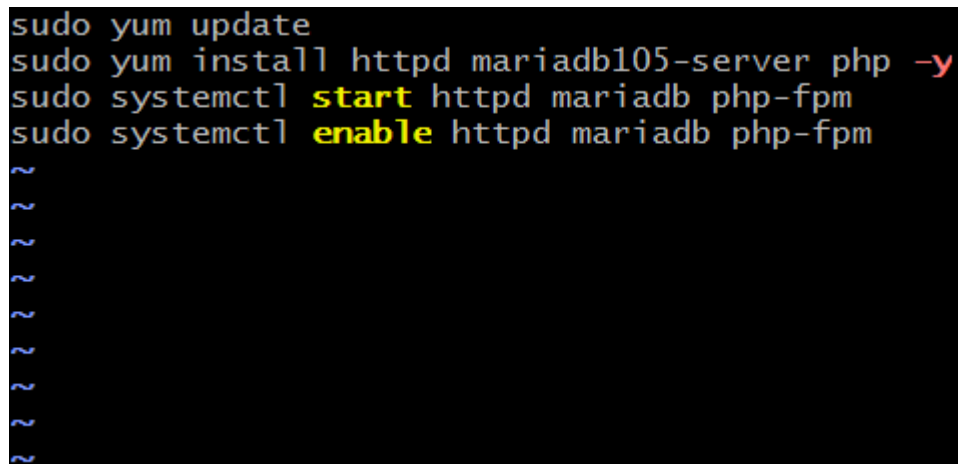
3. Create a LAMP installation script

- Instead of running commands one by one, create a script file Lamp.sh. ``bash

```
sudo vim Lamp.sh
```

- Inside the file, press i (insert mode) and add the following commands:

```
sudo yum update
sudo yum install httpd mariadb105-server php -y
sudo systemctl start httpd mariadb php-fpm
sudo systemctl enable httpd mariadb php-fpm
```

A terminal window with a black background and white text. It shows the execution of the LAMP installation script. The commands are: 'sudo yum update', 'sudo yum install httpd mariadb105-server php -y', 'sudo systemctl start httpd mariadb php-fpm', and 'sudo systemctl enable httpd mariadb php-fpm'. The output of the first command is 'sudo yum update' followed by a series of tilde characters '~'. The output of the second command is 'sudo yum install httpd mariadb105-server php -y' followed by a series of tilde characters '~'. The output of the third command is 'sudo systemctl start httpd mariadb php-fpm' followed by a series of tilde characters '~'. The output of the fourth command is 'sudo systemctl enable httpd mariadb php-fpm' followed by a series of tilde characters '~'.

```
sudo yum update
sudo yum install httpd mariadb105-server php -y
sudo systemctl start httpd mariadb php-fpm
sudo systemctl enable httpd mariadb php-fpm
~
~
~
~
~
~
~
~
~
~
```

- Save and exit:
- Now run the script:

```
sudo bash Lamp.sh
```

```
[ec2-user@ip-172-31-31-36 ~]$ sudo vim Lamp.sh
[ec2-user@ip-172-31-31-36 ~]$ sudo bash Lamp.sh
Amazon Linux 2023 Kernel Livepatch repository 243 kB/s | 26 kB 00:00
=====
WARNING:
  A newer release of "Amazon Linux" is available.

  Available Versions:

  Version 2023.9.20250929:
    Run the following command to upgrade to 2023.9.20250929:

      dnf upgrade --releasever=2023.9.20250929

  Release notes:
    https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.9.20250929.html
=====
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:02 ago on Tue Sep 30 15:07:17 2025.
Dependencies resolved.
=====
Package Architecture Version Repository Size
=====
Installing:
httpd x86_64 2.4.65-1.amzn2023.0.1 amazonlinux 47 k
mariadb105-server x86_64 3:10.5.29-1.amzn2023.0.1 amazonlinux 10 M
php8.4 x86_64 8.4.10-1.amzn2023.0.1 amazonlinux 17 k
=====
```

- This will update your system, install Apache, MariaDB, PHP, and enable them at startup.

4. Install PHP MySQL connector

- WordPress needs the PHP MySQL connector.

```
sudo yum install php8.4-mysqlnd.x86_64
cd /var/www/html
ls
```

```
[ec2-user@ip-172-31-31-36 ~]$ sudo yum install php8.4-mysqlnd.x86_64
Last metadata expiration check: 0:01:08 ago on Tue Sep 30 15:07:17 2025.
Dependencies resolved.
=====
Package Architecture Version Repository Size
=====
Installing:
php8.4-mysqlnd x86_64 8.4.10-1.amzn2023.0.1 amazonlinux 156 k
=====
Transaction Summary
=====
Install 1 Package

Total download size: 156 k
Installed size: 438 k
Is this ok [y/N]: y
Downloading Packages:
php8.4-mysqlnd-8.4.10-1.amzn2023.0.1.x86_64.rpm 4.0 MB/s | 156 kB 00:00
Total 2.1 MB/s | 156 kB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing : 1/1
  Installing : php8.4-mysqlnd-8.4.10-1.amzn2023.0.1.x86_64 1/1
  Running scriptlet: php8.4-mysqlnd-8.4.10-1.amzn2023.0.1.x86_64 1/1
  Verifying : php8.4-mysqlnd-8.4.10-1.amzn2023.0.1.x86_64 1/1
```

5. Download & Extract WordPress

```
sudo wget https://wordpress.org/latest.tar.gz
ls
```

- **wget https://wordpress.org/latest.tar.gz** → Downloads the latest version of WordPress as a compressed .tar.gz file.
- **ls** → Lists all files and directories in the current folder, so you can verify what's downloaded or extracted.

```
[ec2-user@ip-172-31-31-36 htm]$ sudo wget https://wordpress.org/latest.tar.gz
--2025-09-30 15:11:27-- https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 198.143.164.252, 2607:f978:5:8002::c68f:a4fc
Connecting to wordpress.org (wordpress.org)|198.143.164.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26925441 (26M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz          100%[=====>]
2025-09-30 15:11:28 (70.6 MB/s) - 'latest.tar.gz' saved [26925441/26925441]

[ec2-user@ip-172-31-31-36 htm]$
```

```
sudo tar -xvzf latest.tar.gz
ls
```

- **tar -xvzf latest.tar.gz** → Extracts the downloaded WordPress archive.
 - **x** → extract files
 - **v** → verbose (shows progress)
 - **z** → handles gzip compression
 - **f** → specifies the filename

```
sudo rm -rf latest.tar.gz
ls
```

- **rm -rf latest.tar.gz** → Removes the compressed file after extraction to save space.
- Go inside WordPress:

```
cd wordpress/
```

```
ls
```

6. Create the WordPress Database

```
sudo mysql
```

- Inside MySQL:

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';  
CREATE DATABASE wordpressdb;  
SHOW DATABASES;  
EXIT;
```

Explanation:

- **sudo mysql** → Opens the MySQL command-line interface as the root user.
- **ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';** → Sets or resets the root user password to root (you can change it to something more secure).
- **CREATE DATABASE wordpressdb;** → Creates a new database named wordpressdb for the WordPress site.
- **SHOW DATABASES;** → Lists all available databases to confirm that wordpressdb has been created successfully.

- **EXIT;** → Exits the MySQL prompt and returns to the normal terminal.

```
[ec2-user@ip-172-31-31-36 wordpress]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> alter user root@localhost identified by 'root';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> create database wordpressdb;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| wordpressdb |
+-----+
4 rows in set (0.002 sec)

MariaDB [(none)]> exit
Bye
[ec2-user@ip-172-31-31-36 wordpress]$ |
```

7. Change Ownership of WordPress Folder

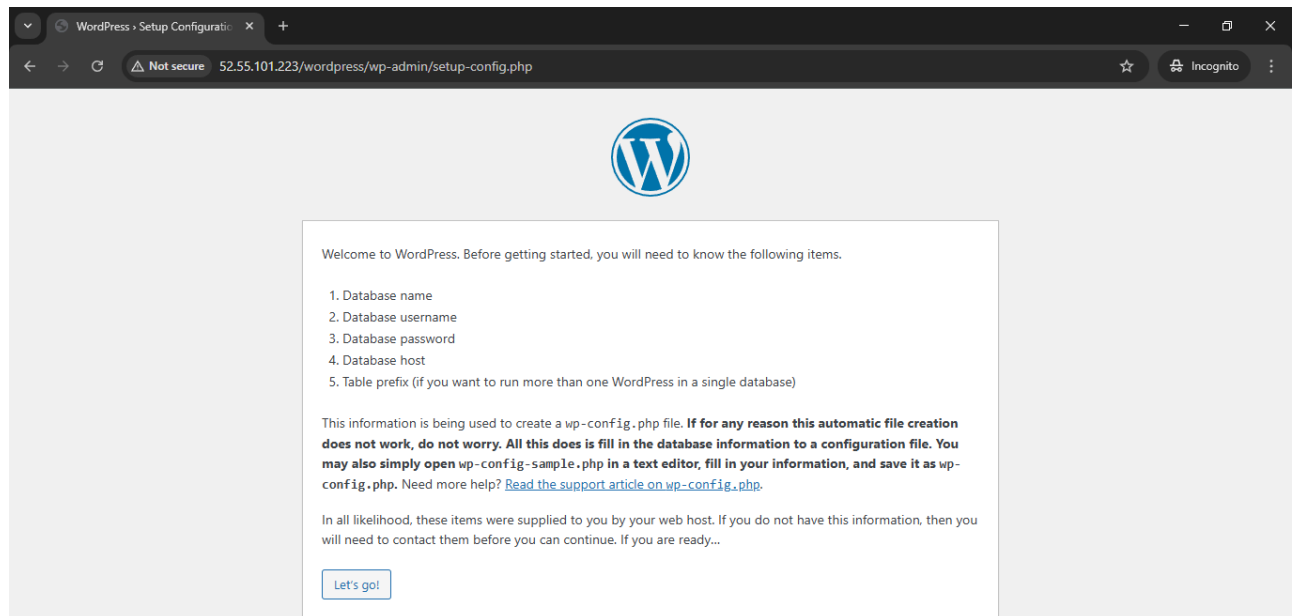
```
cd /var/www/html/
sudo chown -R apache:apache wordpress
sudo systemctl restart httpd mariadb php-fpm
```

- **sudo chown -R apache:apache wordpress** → Changes the ownership of the wordpress folder (and all its files) to the apache user and group.
 - **chown** → change ownership
 - **-R** → applies changes recursively to all files and subdirectories
 - Ensures Apache can read and write WordPress files properly.
- **Sudo systemctl restart httpd mariadb php-fpm** → Restarts the necessary services:

8. Access WordPress Setup in Browser

- **Go to:**

```
http://<PUBLIC_IP>/wordpress
```



- **WordPress Installation Steps:**

1. Database Configuration:

- Database Name: **wordpressdb**
- Username: **root**
- Password: **root**
- Database Host: **localhost**



Below you should enter your database connection details. If you are not sure about these, contact your host.

Database Name
The name of the database you want to use with WordPress.

Username
Your database username.

Password [Show](#)
Your database password.

Database Host
You should be able to get this info from your web host, if localhost does not work.

Table Prefix
If you want to run multiple WordPress installations in a single database, change this.

[Submit](#)

- → Click Submit, then **Run the Installation**.

2. Site Details Setup:

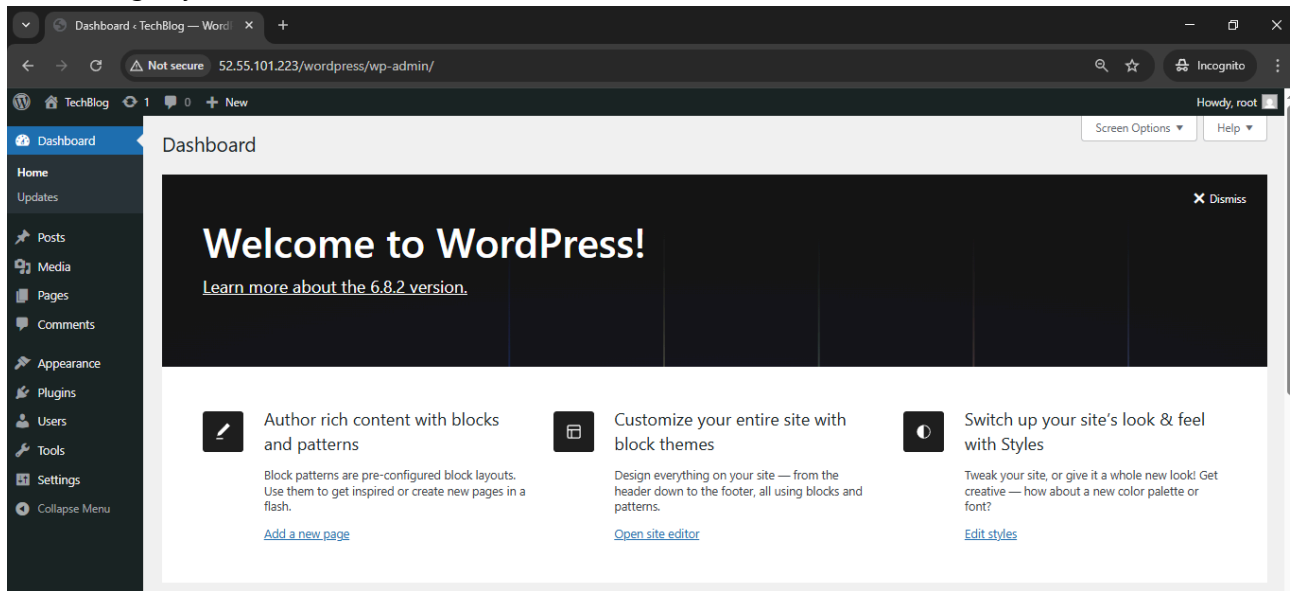
- Site Title: **TechBlog**
- Username: **root**
- Password: **root**
- Email: **Your-Mail**

- → Click **Install** WordPress.

3. Login to Dashboard:

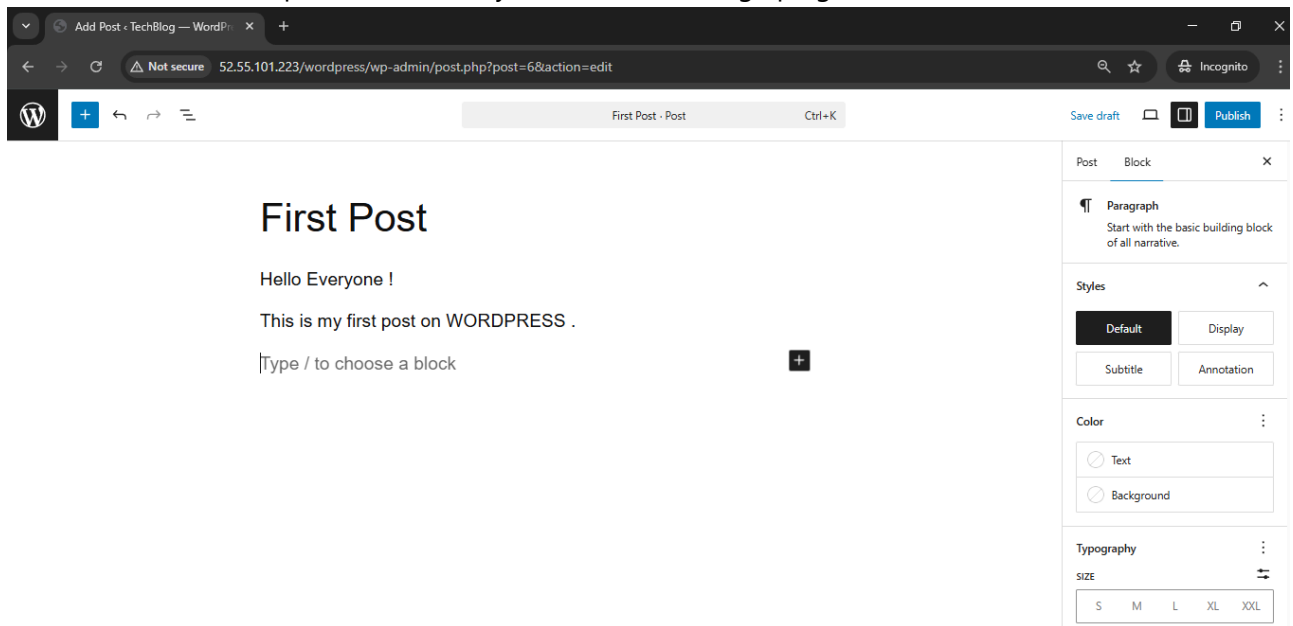
- Username: **root**
- Password: **root**

- After login, you'll be redirected to the WordPress Dashboard — **Welcome to WordPress!**



4. Next Steps:

- You can now add new posts, customize your site, and manage plugins/themes from the dashboard.



Conclusion

- Successfully deployed a WordPress site on AWS EC2 using LAMP stack.
- This setup provides a basic, scalable foundation for hosting dynamic websites.

Troubleshooting

- If WordPress page not loading → Check security group (port 80 open)
- If database error → Verify DB name, username, password
- Restart services:

```
sudo systemctl restart httpd mariadb php-fpm
```

Future Improvements

- Use RDS instead of local MariaDB
- Configure HTTPS using SSL (Let's Encrypt)
- Add a domain name (Route 53)
- Automate using Terraform or Ansible